NATIONAL RESEARCH UNIVERSITY
HIGHER SCHOOL OF ECONOMICS

Faculty of Computer Science
Bachelor's Programme "Data Science and Business Analytics"

Software Project Report on the Topic:

A Smart System for Accounting and Analyzing Supplies, Goods, and Orders for Small Businesses.

Submitted by the Students:
group #БПАД233, 3rd year of study Miniaitsev Ivan Vasilyevich
group #БПАД233, 3rd year of study Storozheva Elizaveta Olegovna

Approved by the Project Supervisor:
Kornelyuk Egor Kirillovich
Teacher
Faculty of Computer Science, HSE University

Moscow 2026

Содержание

Abstract

This project is focused on the development of a software system for accounting and analytics of supplies, goods, and orders for small businesses. The main goal of the project is to help small companies manage their operational data in a centralized way and support decision-making using automated data processing and basic machine learning methods.

Within the project, a software prototype is planned to be developed. The system will include modules for supply management, inventory tracking, and order accounting. It will also support different user access levels and automatic updates of statistical indicators. Special attention will be given to the use of predictive and recommendation machine learning models. These models are planned to be used for demand forecasting, detection of unusual situations in supply chains, and generation of recommendations for inventory management.

The system is expected to have a modular architecture that allows easy extension and scaling. It will also provide a user-friendly interface designed for practical use in small businesses. During the project, existing software solutions will be reviewed and compared in order to justify the advantages of the proposed system in terms of functionality and analytical capabilities.

The expected result of the project is a working software prototype of an intelligent accounting and analytics system, as well as an analytical evaluation of the proposed architectural and machine learning solutions in comparison with existing analogues. The results of this work can be used as a basis for further development of intelligent business management systems for small enterprises.

Аннотация

Этот проект направлен на разработку программной системы для учёта и анализа поставок, товаров и заказов для малого бизнеса. Основная цель проекта — помочь небольшим компаниям централизованно управлять своими операционными данными и принимать решения с помощью автоматизированной обработки данных и базовых методов машинного обучения.

В рамках проекта планируется разработать прототип программного обеспечения. Система будет включать модули для управления поставками, отслеживания запасов и учёта заказов. Она также будет поддерживать различные уровни доступа пользователей и автоматическое обновление статистических показателей. Особое внимание будет уделено использованию моделей машинного обучения для прогнозирования и рекомендаций. Эти модели планируется использовать для прогнозирования спроса, выявления нестандартных ситуаций в цепочках поставок и формирования рекомендаций по управлению запасами.

Ожидается, что система будет иметь модульную архитектуру, которая позволит легко расширять и масштабировать её. Она также будет иметь удобный интерфейс, предназначенный для практического использования в малом бизнесе. В ходе проекта будут рассмотрены и сопоставлены существующие программные решения, чтобы обосновать преимущества предлагаемой системы с точки зрения функциональности и аналитических возможностей.

Ожидаемым результатом проекта является рабочий программный прототип интеллектуальной системы бухгалтерского учёта и аналитики, а также аналитическая оценка предлагаемых архитектурных решений и решений на основе машинного обучения в сравнении с существующими аналогами. Результаты этой работы могут быть использованы в качестве основы для дальнейшей разработки интеллектуальных систем управления бизнесом для малых предприятий.

Keywords

- Software system
- Small business analytics
- Inventory management
- Supply chain accounting
- Order management
- Access control
- Predictive modeling
- Machine learning

1 Introduction

Small businesses often face serious difficulties in managing supplies, goods, and customer orders. In many cases, accounting processes are partially automated or handled using simple tools such as spreadsheets. This leads to data fragmentation, errors in inventory tracking, lack of up-to-date analytics, and weak support for management decisions. These problems become especially critical when the business grows and the volume of operational data increases.

Modern information systems offer a wide range of solutions for inventory and order management. However, many existing systems are designed for medium or large enterprises and are too complex, expensive, or difficult to adapt for small businesses. As a result, small companies often do not use advanced analytical tools and rely mainly on manual analysis and intuition when making decisions related to supply planning, inventory levels, and order processing.

At the same time, recent advances in data analytics and machine learning make it possible to improve operational efficiency even with relatively small amounts of data. Predictive models can be used to forecast demand, identify potential shortages or overstocking, and detect unusual patterns in supply chains. Recommendation systems can support decision-making by suggesting optimal actions based on historical data. Despite this potential, such methods are still rarely integrated into simple and affordable software solutions for small businesses.

The goal of this project is to develop an intelligent software system for accounting and analytics of supplies, goods, and orders, specifically oriented toward the needs of small businesses. The proposed system aims to combine basic accounting functionality with automated analytics and machine learning methods in a single unified platform. The project focuses on practical applicability, simplicity of use, and modular system architecture.

The developed system is planned to include several key components. These components

include modules for supply accounting, inventory management, and order tracking, as well as a role-based access control system for different categories of users. The system will also provide automatically updated statistical indicators that reflect the current state of business operations. In addition, predictive and recommendation models are planned to be integrated to support demand forecasting and inventory management decisions.

The relevance of this project is determined by the growing need for affordable and flexible digital tools that help small businesses manage data and make informed decisions. The innovative aspect of the project lies in the integration of machine learning models into a lightweight accounting system that is adapted to real-world constraints of small enterprises. Unlike many existing solutions, the proposed system is designed as a modular prototype that can be extended and adapted to different business scenarios.

The expected outcome of the project is a functional software prototype and an analytical evaluation of the selected architectural, analytical, and machine learning solutions. The results of this work are currently planned and will be obtained during the further stages of the project. In the following sections of the report, an overview of existing software solutions will be provided, followed by a detailed description of the system architecture, implementation choices, and planned evaluation methods.

1.1 Problem Statement

This project addresses the absence of practical digital tools that effectively support monitoring and analysis of product flows and customer requests in small enterprises. Available applications are often either too limited in functionality or overly complex, which makes them unsuitable for everyday operational use in small-scale business environments.

Simplified solutions usually focus on data registration and static reporting, offering little support for analytical insights or forward-looking assessment. In contrast, platforms that apply advanced data-driven techniques are frequently difficult to deploy, require significant resources, and lack clear integration with operational workflows, reducing their transparency and practical value.

Consequently, there is a demand for a unified solution that combines operational tracking, dynamic analytical processing, and data-driven decision support within a flexible and modular software environment. Such an approach should enable efficient daily operations while remaining accessible and adaptable to the constraints of small businesses.

1.2 Project Goal and Objectives

The goal of this project is to design and implement a software system that supports operational tracking and analytical processing of supplies, inventory items, and customer orders for small businesses, with integrated data-driven decision support.

To accomplish this goal, the following objectives are defined:

- to study currently used digital tools for managing operational business data and to identify gaps in their analytical and functional capabilities;
- to propose a system design that ensures modularity, controlled user permissions, and extensibility of analytical components;
- to construct data analysis and forecasting models that support planning and optimization of inventory-related activities;
- to build a working prototype that combines operational data handling, automated statistics, and analytical support features;
- to validate the developed solution by applying suitable evaluation criteria related to efficiency, usability, and applicability in real business scenarios, including comparison with existing solutions.

1.3 Expected Outcomes

As a result of this project, a functional software prototype for managing and analyzing business operations is expected to be developed. The system is planned to demonstrate integrated operational tracking, automated statistical analysis, and data-driven support for decision-making.

In addition, a comparative evaluation of the proposed solution with existing software tools is expected, highlighting its practical advantages and limitations.

2 Overview of Bibliographical Sources

This section examines relevant studies and technical sources focusing on operational data management, business analytics, and web-oriented software solutions for small businesses. The reviewed materials provide methodological and applied support for the design and implementation of the proposed system.

2.1 Warehouse inventory management system using IoT and open source framework

From this article, this project adopts the idea of centralized storage of detailed information about goods to simplify search and tracking processes. The concept of linking physical items with structured digital records is used as a basis for inventory monitoring. The importance of real-time or near real-time data updates is taken into account to ensure current analytical information. The separation between material flow and information flow is considered when designing the system architecture. In addition, the focus on cost-efficient and scalable solutions influences the choice of technologies and overall system design. [4]

2.2 Comparative Analysis on Front-End Frameworks for Web Applications

From here, the project adopts the idea of using modern web frameworks to ensure structured, maintainable, and scalable application development. The component-based approach is taken as a foundation for building reusable and extensible user interface elements. The importance of clear architectural separation between interface logic and application functionality is reflected in the system design. The role of frameworks as a means to accelerate development and enforce best practices is considered during technology selection. In addition, the focus on usability and adaptability of web applications influences the choice of frontend solutions for the project. [5]

2.3 WEB APPLICATION DEVELOPMENT: CHALLENGES AND THE ROLE OF WEB ENGINEERING

From this work, the project adopts the view of the Web as a primary platform for deploying complex business information systems. The importance of treating web applications as full-scale software systems rather than simple web pages is reflected in the chosen development approach. Special attention is given to system-level requirements such as reliability, scalability, and maintainability. The need for structured design methodologies is taken into account when planning the architecture and development process. In addition, the tight integration between web applications and backend data systems influences the overall system design of the project. [3]

2.4 Machine Learning for Developers

From this material, the project adopts the view of data as the main resource for building intelligent system functionality. The idea that learning systems improve performance through experience and measurable criteria is reflected in the design of analytical components. The separation of tasks, data, and evaluation metrics is used when planning machine learning integration. The focus on practical applicability of machine learning methods guides the selection of relatively simple and interpretable models. In addition, the need to balance analytical power with accessibility for non-expert users is considered in the system design. [1]

2.5 A Comprehensive Review of Security Measures in Database Systems: Assessing Authentication, Access Control, and Beyond

From this study, the project adopts the importance of structured user authentication and role-based permission management for protecting stored data. The need to control access to information at different system levels is reflected in the design of access mechanisms. Consideration of data protection techniques influences decisions related to sensitive information

handling. The relevance of monitoring and logging system actions is taken into account to support accountability and system reliability. In addition, awareness of modern deployment environments informs security-related design choices. [2]

Overall, the examined works emphasize the role of analytical methods in supporting business processes and the need for structured system design for real-world applications. This project follows these ideas by incorporating data analysis techniques into a modular web-oriented solution for operational management.

3 Plan of Further Work

This section outlines the planned stages of the project implementation, including an approximate timeline and distribution of responsibilities between the two team members. The project is planned to be developed incrementally, from problem analysis and system design to implementation, evaluation, and final documentation.

3.1 Stage 1: Domain Analysis and Review of Existing Solutions

February 1 – February 20

At this stage, the problem domain of operational accounting and analytics for small businesses will be explored. Existing software products for inventory, supply, and order management will be reviewed in order to identify common approaches, limitations, and unmet needs. The results of this analysis will serve as a conceptual foundation for further system design.

- Miniaitsev Ivan: analysis of machine learning methods for learner modeling and task personalization, identification of backend-related challenges.
- Storozheva Elizaveta: analysis of existing web-based educational platforms, user interface approaches, and data storage solutions.

3.2 Stage 2: Requirement Definition and Use Case Description

February 21 – March 5

During this stage, the functional and non-functional requirements of the proposed system will be formulated. Key user roles, interaction scenarios, and system constraints will be defined to ensure clarity of expected system behavior.

- Miniaitsev Ivan: formulation of backend requirements, API-level functionality, and personalization logic requirements.
- Storozheva Elizaveta: formulation of frontend requirements, user interaction scenarios, and database-related constraints.

The outcome of this stage will be a structured system requirement specification.

3.3 Stage 3: Database and Data Model Design

March 6 – March 20

This stage focuses on designing the data structures required to support operational tracking and analytics. Logical data entities, relationships, and integrity constraints will be defined. Data preparation needs for analytical processing will also be considered.

- Miniaitsev Ivan: design of feature sets, preprocessing pipelines, and data interfaces for machine learning models.
- Storozheva Elizaveta: design of database schema, data storage formats, and data access mechanisms.

3.4 Stage 4: System Architecture Design

Timeframe: March 21 – April 5

At this stage, the overall architecture of the software system will be designed. The system will be decomposed into core components, including data storage, application logic, analytical modules, and user interface. Interactions between components will be defined.

- Miniaitsev Ivan: design of backend services, machine learning integration, and API specifications.
- Storozheva Elizaveta: design of frontend architecture, database structure, and frontend–backend communication.

The result of this stage will be a complete architectural description of the system.

3.5 Stage 5: Development of Analytical and Predictive Models

April 6 – April 25

During this stage, data analysis and predictive models will be developed and tested. These models will focus on demand estimation, trend identification, and support for inventory-related decisions.

- Miniaitsev Ivan: implementation, training, and tuning of machine learning models, as well as preparation of inference logic for backend integration.
- Storozheva Elizaveta: preparation of data access queries and support of experimental evaluation from the system side.

3.6 Stage 6: Prototype Implementation

April 26 – May 10

Based on the defined architecture, a functional software prototype will be implemented. Core system features, access control mechanisms, and analytical components will be integrated into a single working solution.

- Miniaitsev Ivan: implementation of backend logic, integration of machine learning models, and realization of personalization mechanisms.
- Storozheva Elizaveta: implementation of frontend interface, user interaction logic, and database integration.

3.7 Stage 7: Integration, Testing, and Evaluation

May 11 – May 20

At this stage, the complete system will be tested to verify correctness, consistency, and basic performance. Analytical functionality and system stability will be evaluated using predefined criteria.

- Miniaitsev Ivan: backend testing, validation of machine learning integration, and debugging.
- Storozheva Elizaveta: frontend testing, database consistency checks, and user interface validation.

3.8 Stage 8: Final Documentation and Report Preparation

May 21 – May 31

The final stage includes preparation of the project report and supporting documentation. All design decisions, implementation details, and evaluation results will be summarized and formatted according to academic requirements.

- Miniaitsev Ivan: quantitative analysis of machine learning models and personalization quality.
- Storozheva Elizaveta: analysis of system performance, usability evaluation, and result visualization.

4 System Requirements Specification

This section defines the functional and non-functional requirements of the proposed web-based analytical system for small businesses. The requirements describe the expected system behavior, constraints, and quality attributes.

4.1 Functional Requirements

Functional requirements specify the operations and services that the system must provide.

4.1.1 User Authentication and Account Management

The system shall:

1. Allow users to access the web application via a standard web browser.
2. Provide user registration functionality.
3. Allow users to log in using valid credentials.
4. Allow users to log out securely.
5. Display informative error messages in case of invalid authentication or registration input.
6. Provide onboarding information after successful account creation.
7. Allow users to view and edit personal profile information.
8. Allow users to define and modify company-related information.
9. Allow users to upload and update a profile image.

4.1.2 Dashboard Management

The system shall:

1. Provide access to the main dashboard page after successful authentication.
2. Display analytical dashboards containing business performance indicators.
3. Allow users to create, modify, and delete dashboard configurations.
4. Enable navigation back to the main dashboard from internal system pages.

4.1.3 Data Management

The system shall:

1. Allow users to upload business data files in supported formats.
2. Maintain a list of previously uploaded datasets.
3. Display uploaded data in structured tabular form.
4. Validate uploaded data and display error messages in case of inconsistencies.
5. Indicate processing status of uploaded datasets.

4.1.4 Employee and Access Management

The system shall:

1. Allow viewing of registered company employees.
2. Allow addition of new employees.
3. Allow removal of employees.
4. Provide role-based access control mechanisms for managing employee permissions.

4.1.5 Support and Information Services

The system shall:

1. Provide access to a support information page.
2. Enable users to submit support requests via email.
3. Allow submission of feedback and improvement suggestions.
4. Provide an informational page describing the application and its authors.

4.2 Non-Functional Requirements

Non-functional requirements define system quality attributes and operational constraints.

4.2.1 Performance Requirements

1. The system shall load primary interface pages within three seconds under normal operating conditions.
2. The system shall process uploaded data files of up to 10 MB within an acceptable processing time.
3. Dashboard visualizations shall render without significant delay.

4.2.2 Security Requirements

1. User passwords shall be securely stored using cryptographic hashing mechanisms.
2. The system shall enforce authentication and role-based access control.
3. Users shall only access their own datasets and dashboards.
4. File uploads shall be validated to prevent malicious content execution.
5. Sensitive data shall not be exposed in error messages or system responses.

4.2.3 Usability Requirements

1. The system interface shall provide intuitive navigation.
2. Error messages shall be clear and informative.
3. The interface shall follow a consistent visual design.
4. The system shall provide guidance for first-time users.
5. The application shall not require advanced technical knowledge.

4.2.4 Reliability Requirements

1. Uploaded data shall be stored persistently in the database.
2. The system shall prevent data loss during file upload.
3. The application shall handle incorrect user input without system failure.
4. System failures shall not compromise data integrity.

4.2.5 Compatibility Requirements

1. The application shall function correctly in modern web browsers.
2. The system shall support common spreadsheet file formats including XLSX and XLS.

4.2.6 Scalability Requirements

1. The system architecture shall support future functional expansion.
2. The database structure shall allow handling multiple users and datasets simultaneously.

5 Technology Stack Selection and Justification

5.1 Analysis of Web Application Development Approaches

During the implementation stage, an analysis of possible approaches to web application development was conducted in order to determine the most appropriate technological solution for the project.

Several development strategies were considered.

The first approach involves the use of low-code and no-code platforms. Although such tools enable rapid interface creation, they provide limited flexibility for implementing complex business logic and analytical data processing. For a system designed to process user-uploaded datasets and generate analytical dashboards, this approach does not ensure sufficient extensibility.

The second approach is based on modern client-side frameworks combined with a separate backend application programming interface. While this architecture supports interactive interfaces, it significantly increases structural complexity and requires additional integration between frontend and backend components.

The third approach involves server-side web frameworks that integrate application logic, database management, and user interface generation within a unified architecture. This approach enables efficient development of structured systems with authentication, data storage, and analytical processing capabilities.

Considering the project objectives, a server-side framework was selected as the most appropriate architectural solution.

5.2 Choice of Programming Language

Python was selected as the primary programming language for the project for the following reasons:

- strong support for data analysis and machine learning;
- availability of mature libraries for processing structured data;
- clear and maintainable syntax;
- compatibility between analytical modules and web application logic.

The use of Python enables integration of data processing and web development within a single technological environment and reduces architectural fragmentation.

5.3 Choice of Django Framework

The Django framework was selected as the core technology for web application development based on the following factors:

- built-in authentication and role-based access control mechanisms;

- integrated object-relational mapping system for structured database management;
- high level of built-in security against common web vulnerabilities;
- modular architecture supporting scalable system design;
- rapid prototyping capabilities for structured web applications.

Django provides a stable and extensible foundation that supports both operational data management and future analytical module integration.

5.4 Final Technological Decision

Based on the conducted analysis, the system is implemented as a server-side web application using Python and Django. The selected stack ensures consistency between analytical processing and web functionality, supports secure data handling, and allows further architectural expansion.

6 Requirements Implementation and Design Validation

This section demonstrates the correspondence between the defined system requirements and the developed interface prototypes. Each functional requirement is validated through the designed system pages.

6.1 Validation of User Authentication and Account Management Requirements

Welcome Page

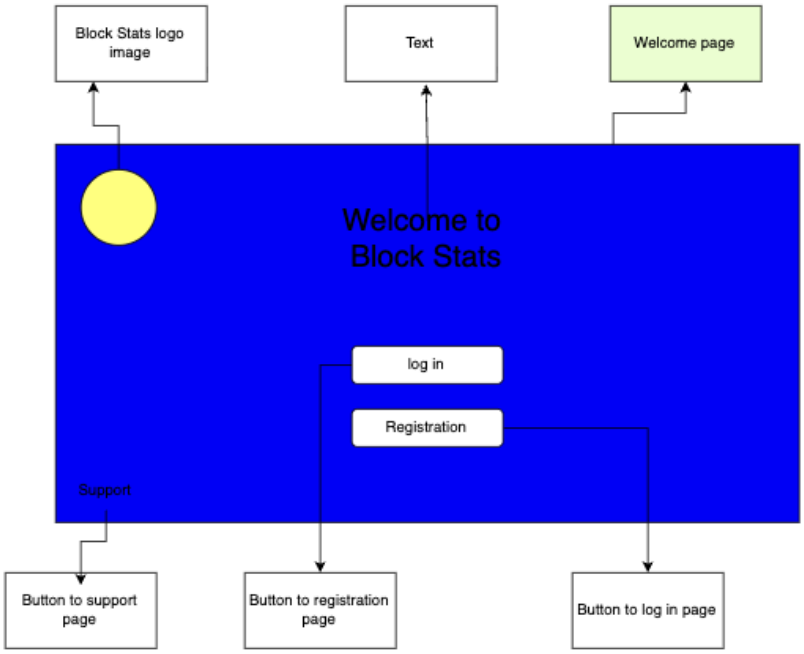

Рис. 1: Welcome Page Interface

The Welcome page satisfies the following requirements:

- 4.1.1.1 – Access to the web application via browser.
- 4.1.1.2 – Navigation to the registration page.
- 4.1.1.3 – Navigation to the login page.
- 4.1.1.6 – Structural support for onboarding information.

Login Page

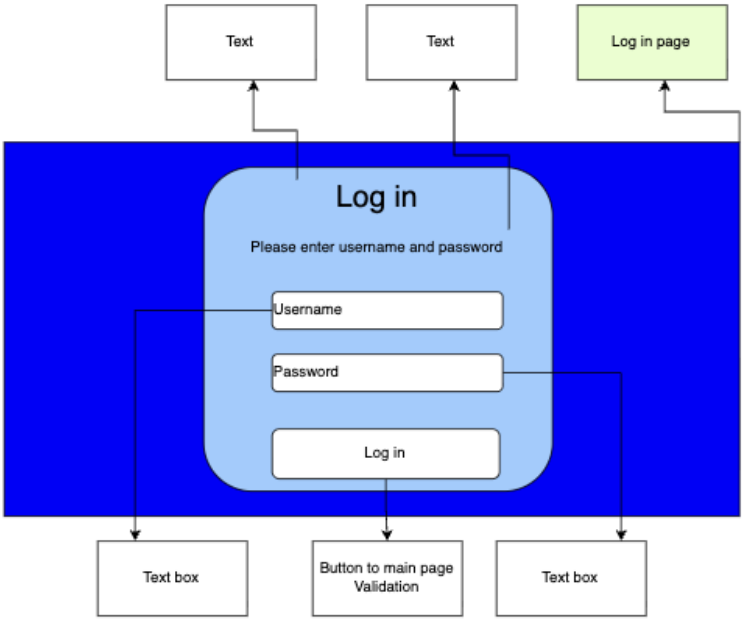

Рис. 2: Login Page Interface

The Login interface satisfies:

- 4.1.1.3 – Authentication using valid credentials.
- 4.1.1.5 – Error message display for incorrect input.
- 4.1.1.4 – Secure logout through authenticated session management.

Registration Page

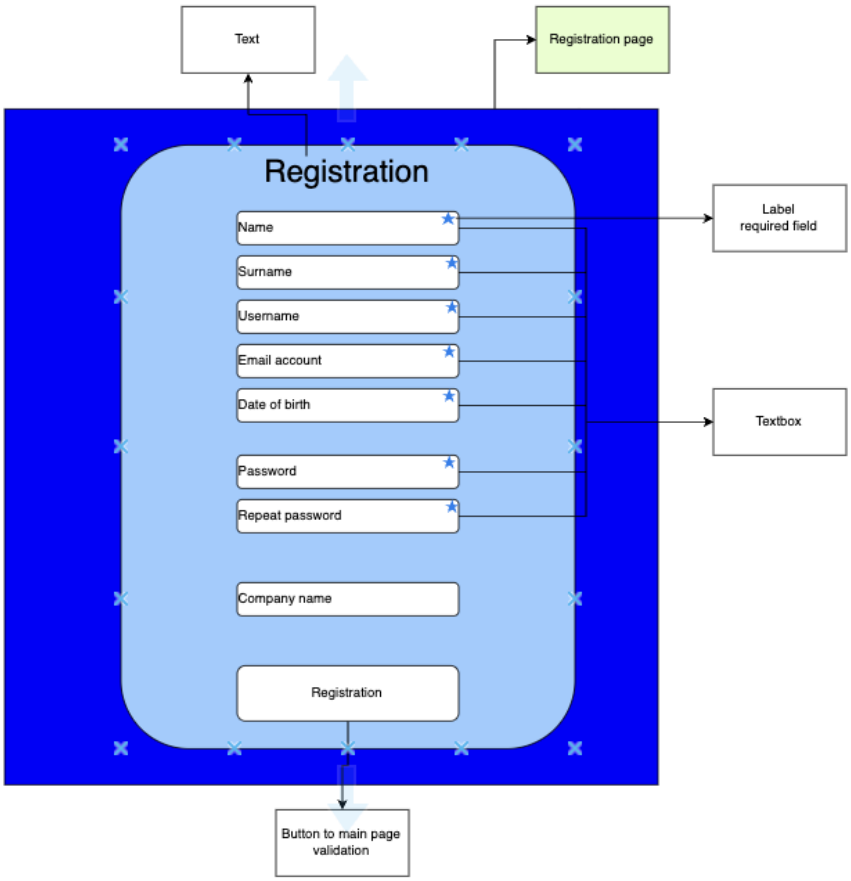

Рис. 3: Registration Page Interface

The Registration page implements:

- 4.1.1.2 – User registration functionality.
- 4.1.1.5 – Required field validation and password confirmation.
- 4.1.1.8 – Collection of company-related information.

Settings Page

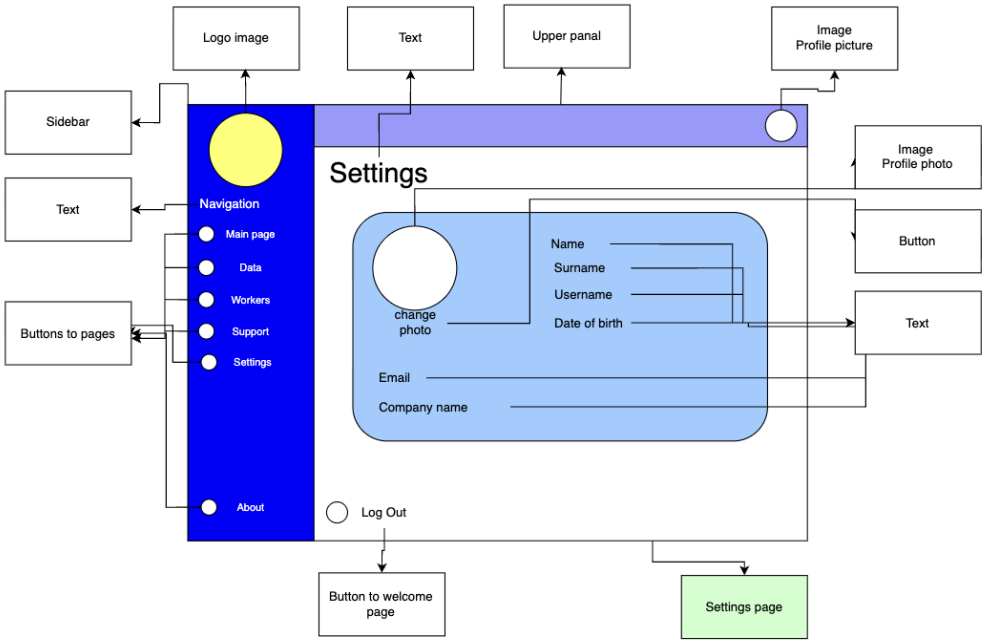

Рис. 4: User Profile and Settings Interface

The Settings page satisfies:

- 4.1.1.7 – Viewing and editing personal profile information.
- 4.1.1.8 – Modification of company-related information.
- 4.1.1.9 – Uploading and updating profile image.
- 4.1.1.4 – Secure logout functionality.

6.2 Validation of Dashboard Management Requirements

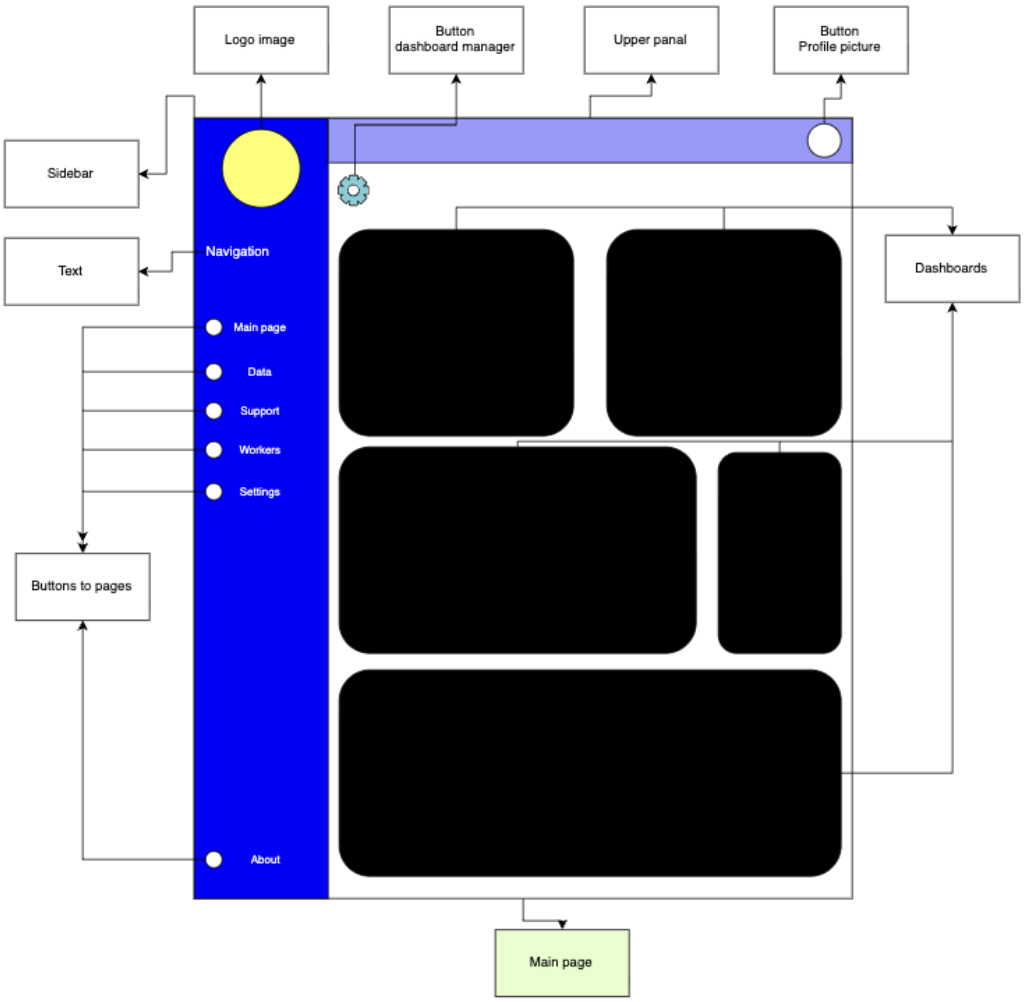

Рис. 5: Main Dashboard Interface

The Main Dashboard page satisfies:

- 4.1.2.1 – Access after successful authentication.
- 4.1.2.2 – Display of analytical dashboard components.
- 4.1.2.4 – Navigation back to main page from internal sections.

The modular dashboard blocks are designed to support creation and modification of dashboard configurations as required by 4.1.2.3.

6.3 Validation of Data Management Requirements

Data Overview Page

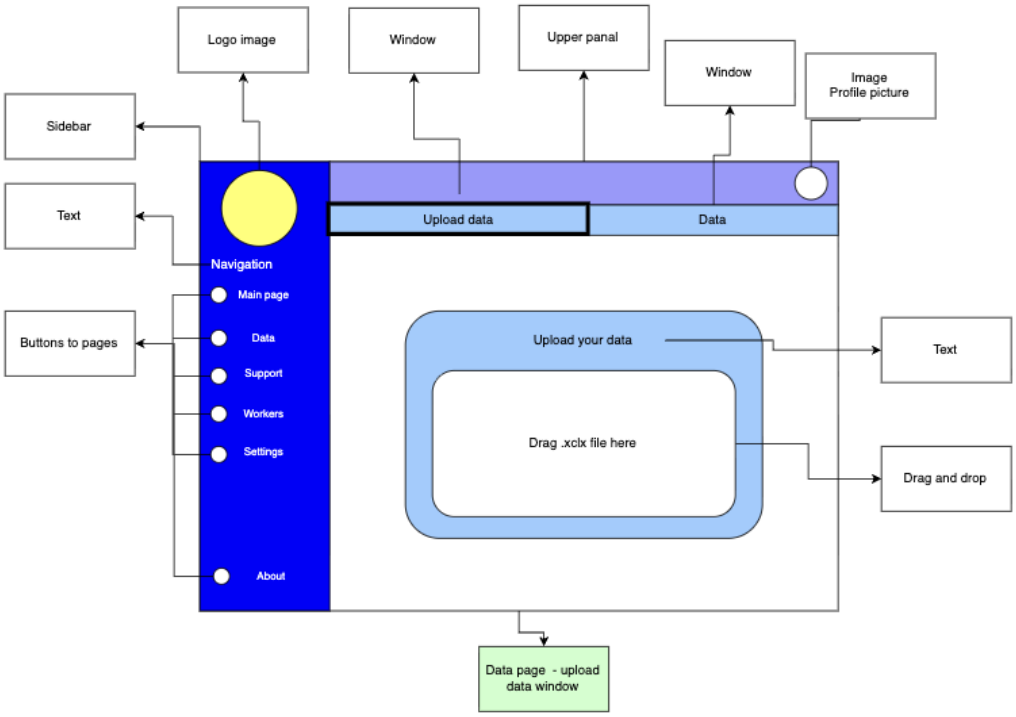

Рис. 6: Data Overview Page

Upload Data Interface

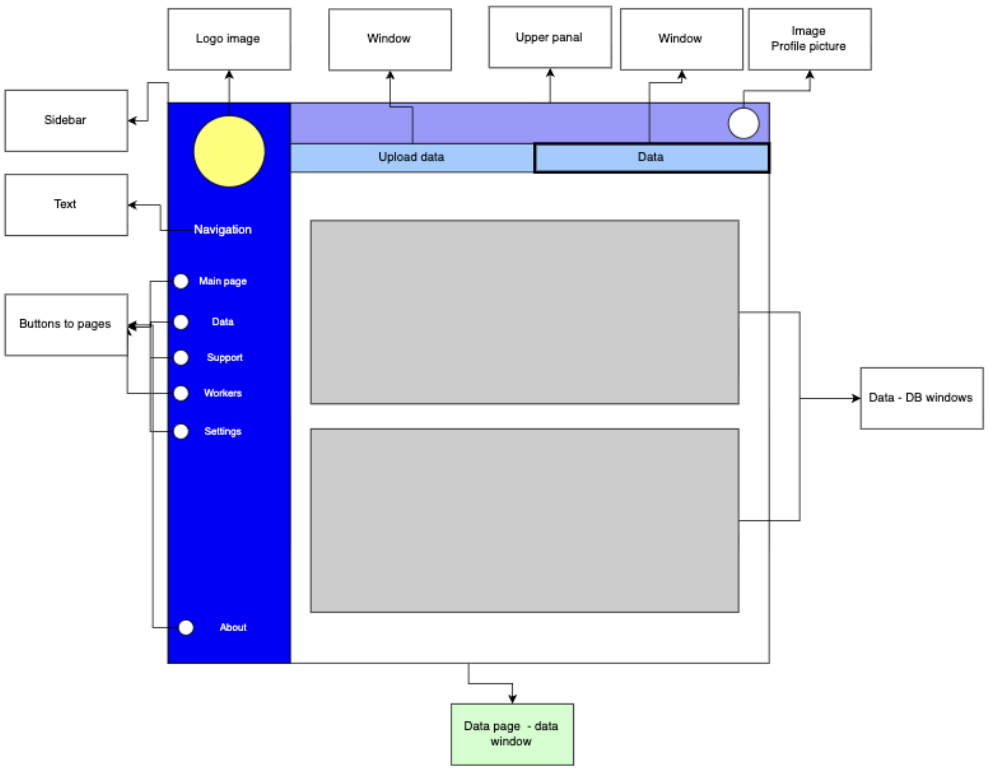

Рис. 7: Data Upload Interface

The Data pages satisfy:

- 4.1.3.1 – Upload of business data files.
- 4.1.3.2 – Display of stored datasets.
- 4.1.3.3 – Structured tabular data presentation.
- 4.1.3.4 – Validation and error indication for incorrect data.
- 4.1.3.5 – Visual representation of processing status.

6.4 Validation of Employee and Access Management Requirements

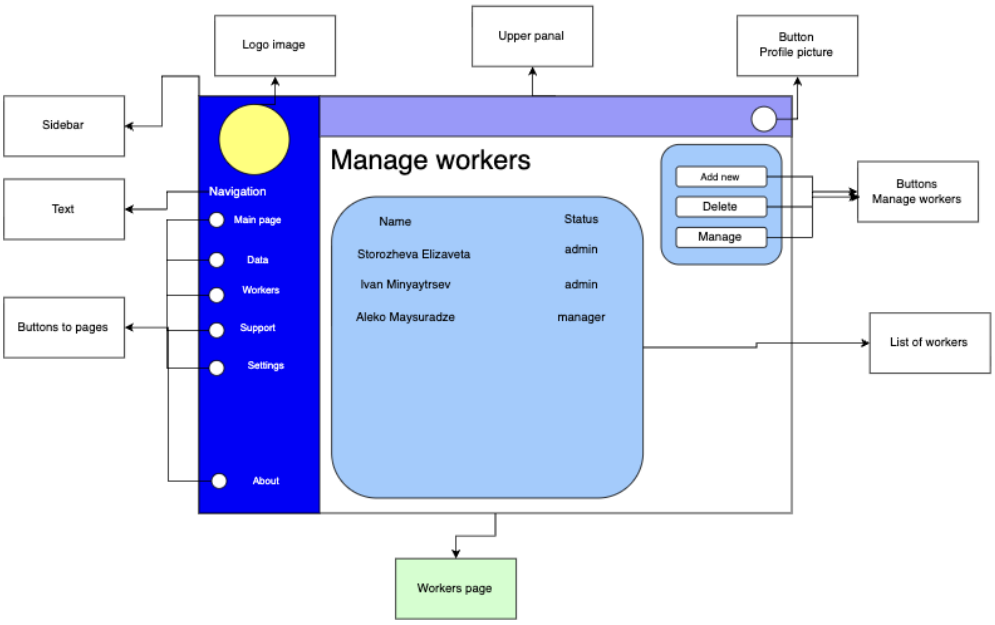

Рис. 8: Employee Management Interface

The Workers page satisfies:

- 4.1.4.1 – Viewing registered employees.
- 4.1.4.2 – Adding new employees.
- 4.1.4.3 – Removing employees.
- 4.1.4.4 – Role-based access control structure.

6.5 Validation of Support and Information Services

Support Page

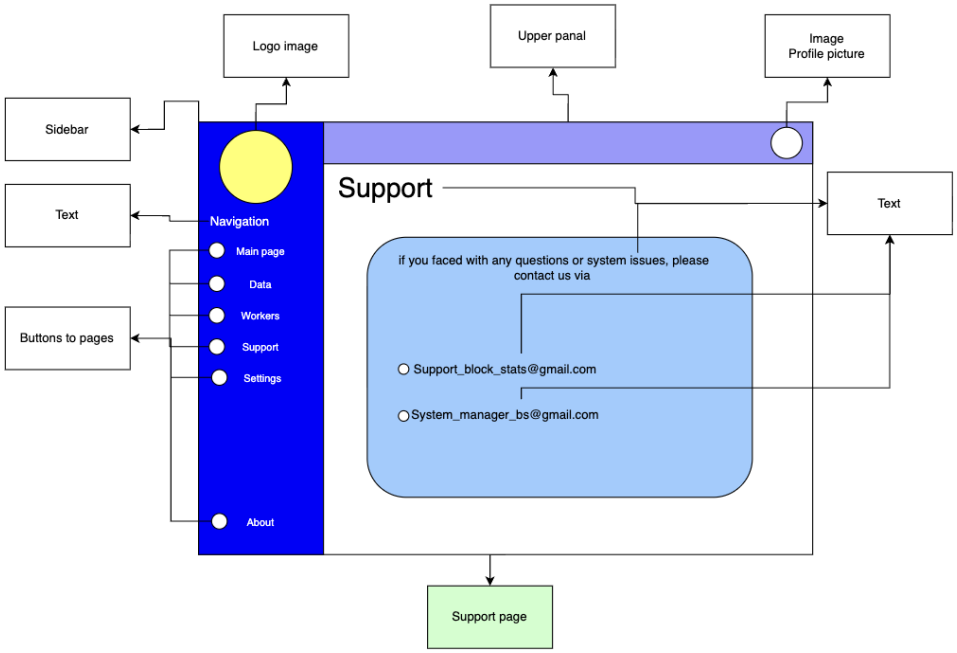

Рис. 9: Support Page Interface

About Page

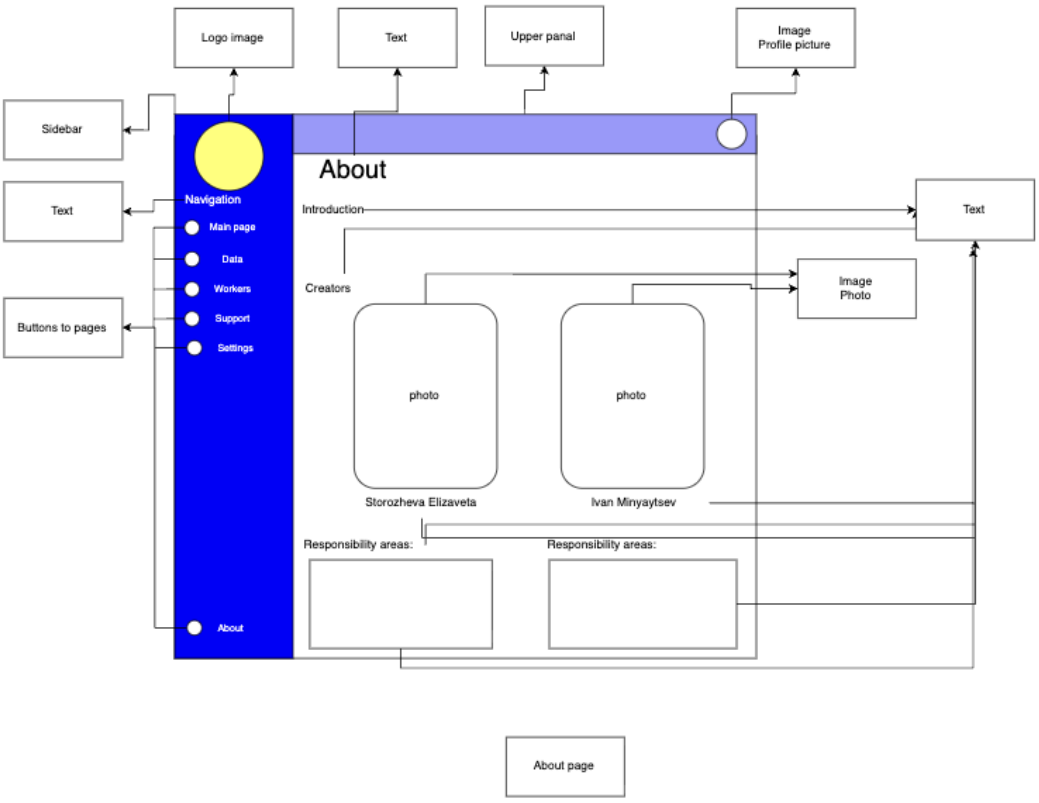

Рис. 10: About Page Interface

These pages satisfy:

- 4.1.5.1 – Access to support information.
- 4.1.5.2 – Contact via email.
- 4.1.5.3 – Feedback and issue reporting.
- 4.1.5.4 – Informational page about the application and authors.

6.6 Non-Functional Requirements Compliance

The developed design also supports the defined non-functional requirements:

- Performance: Lightweight page structure ensures fast loading.
- Security: Authentication, password hashing, and role-based control are supported by backend architecture.
- Usability: Consistent navigation sidebar across all pages.
- Reliability: Structured data validation mechanisms.
- Compatibility: Web-based implementation compatible with modern browsers.
- Scalability: Modular page structure allows future functional expansion.

Conclusion

The developed interface prototypes systematically implement the defined functional requirements and structurally support the non-functional constraints. The traceability between requirement specification and design implementation confirms architectural consistency and requirement compliance.

7 User Interaction Scenario

This section presents a high-level user interaction scenario of the developed web application. The diagram illustrates the logical flow of user actions within the system, including authentication, data upload, validation, and dashboard generation.

7.1 User Flow Overview

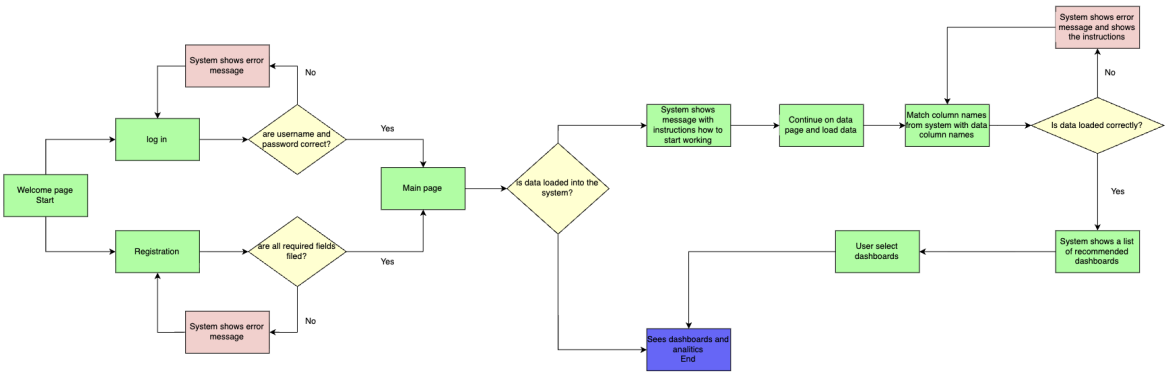

Рис. 11: High-Level User Interaction Flow

The diagram demonstrates the complete interaction cycle beginning from the Welcome page and continuing through authentication or registration.

After successful login, the system checks whether business data has been uploaded. If no data is available, the user is guided to upload a dataset. The system then validates the file, matches column names, and verifies correctness. In case of inconsistencies, informative error messages are displayed.

If validation succeeds, the system generates recommended dashboards. The user selects dashboards and proceeds to view analytical results.

Analysis and Validation of Requirements

The presented interaction flow confirms that:

- Authentication and registration mechanisms are integrated into the workflow.
- Data validation and error handling are implemented as decision points.
- Dashboard generation is dependent on successful data processing.
- The system provides clear navigation between pages.

Conclusion

The user interaction scenario validates the logical coherence of the system architecture and demonstrates alignment between functional requirements and implemented prototype

behavior.

The flow confirms that the system supports structured user navigation, controlled access, data validation procedures, and analytical output generation.

8 Prototype Implementation

This section describes the implemented prototype of the web application and explains its structural organization. The prototype represents a functional server-side web system developed using the Django framework and demonstrates the implementation of core system requirements.

8.1 Project Structure and Architectural Organization

The prototype is organized as a standard Django project consisting of a main configuration module and multiple application modules. The structure follows the Model–View–Template architectural pattern, ensuring separation of concerns between data logic, business logic, and user interface components.

Root Project Directory

The root directory contains the following key components:

- **manage.py** – command-line utility used for running the development server, applying database migrations, and managing administrative tasks.
- **db.sqlite3** – development database used for persistent storage of user accounts and application data.
- **.venv** – virtual environment containing project dependencies.

Smart_Analytics Project Configuration Module

The `Smart_Analytics` directory contains global configuration files:

- **settings.py** – project configuration including database settings, installed applications, static files configuration, and security parameters.
- **urls.py** – central URL routing configuration connecting application-level routes.
- **asgi.py** and **wsgi.py** – entry points for deployment environments.

This module defines global system behavior and integrates individual application components.

Application Modules

The prototype currently consists of two main Django applications:

1. main Application The `main` application implements core system functionality related to dashboards and internal pages.

It contains:

- **models.py** – database models defining system entities.

- **views.py** – business logic and request handling.
- **urls.py** – routing for internal system pages.
- **templates/main/** – HTML templates for main system pages.
- **static/main/** – static resources including CSS and JavaScript files.

This module is responsible for authenticated user functionality and dashboard structure.

2. welcome Application The `welcome` application manages public-facing and authentication-related pages.

It includes:

- **templates/welcome/** – HTML templates for Welcome, Login, Registration, and Support pages.
- **static/welcome/** – CSS and image assets specific to authentication interfaces.
- **views.py** – request handling logic for registration and login processes.
- **urls.py** – routing configuration for authentication pages.

This modular separation improves maintainability and allows independent development of authentication and business logic components.

Template and Static File Organization

The project follows Django best practices by separating:

- **Templates** – HTML files organized per application.
- **Static files** – CSS, images, and JavaScript stored in application-specific static directories.

Such structure ensures scalability and simplifies future extension of interface components.

Architectural Considerations

The prototype adheres to the following architectural principles:

- Separation of concerns via the Model–View–Template pattern.
- Modular application structure for functional separation.
- Reusable layout templates for consistent user interface.
- Configurable routing through centralized URL management.

The implemented structure supports further expansion of analytical modules, database schema extension, and integration of advanced processing components.

8.2 Welcome Page Implementation

The Welcome page represents the public entry point of the system and is implemented within a separate Django application named `welcome`. The separation of this module from the main analytical functionality ensures architectural clarity and modularity.

The page is responsible for initial user interaction and navigation to authentication-related functionality. It implements the entry requirements defined in Section 4.1.1.

Template Structure

The Welcome page is implemented as a Django template located in:

`welcome/templates/welcome/welcome.html`

The template utilizes Django template tags for static file management and URL routing.

Листинг 1: Fragment of welcome.html Template

```
{% load static %}

<img class="logo"
     src="{% static 'welcome/img/logo_welc.png' %}"
     alt="Block Smart logo" />

<a class="support-left"
   href="{% url 'support' %}">
   Support
</a>

<div class="actions">
    <a class="btn"
       href="{% url 'registration' %}">
       Registration
    </a>

    <a class="btn"
       href="{% url 'log_in' %}">
       Log in
    </a>
</div>
```

The use of the `{ % static % }` tag ensures correct resolution of static assets, while the `{ % url % }` tag guarantees centralized URL routing through Django's URL dispatcher.

Navigation Logic

The navigation buttons connect the Welcome page with:

- Registration page
- Login page

- Support page

Routing is handled via the `urls.py` configuration within the `welcome` application, ensuring clean separation between interface and routing logic.

Interface Design Considerations

The page layout is structured using:

- CSS variables for centralized color management
- Responsive layout via media queries
- Structured button styling with hover and active state transitions

The primary design goals were:

- Visual clarity and minimal cognitive load
- Immediate access to authentication functionality
- Consistent branding through reusable design elements

Requirements Coverage

The Welcome page directly satisfies the following functional requirements:

- 4.1.1.1 – Allow users to access the web application via a standard web browser.
- 4.1.1.2 – Provide navigation to registration functionality.
- 4.1.1.3 – Provide navigation to login functionality.
- 4.1.5.1 – Provide access to support information page.

Additionally, the page contributes to the following non-functional requirements:

- 4.2.3.1 – Intuitive navigation.
- 4.2.3.3 – Consistent visual design.
- 4.2.1.1 – Fast loading due to lightweight static implementation.

Architectural Significance

The Welcome module demonstrates:

- Proper use of Django template architecture
- Separation of presentation and routing logic
- Modular application design

This structure ensures extensibility and supports future enhancement of onboarding and public-facing informational components.

8.3 Login Page Implementation

The Login page is implemented within the `welcome` application and provides user authentication functionality. The page is responsible for collecting user credentials, ensuring secure form submission, validating input, and initiating an authenticated session when credentials are correct.

Template Implementation

The Login page template is located at:

`welcome/templates/welcome/log_in.html`

The interface is implemented as an HTML form with secure submission and error message handling. Static styling is connected via a dedicated CSS file located in `welcome/static/welcom`

Листинг 2: Fragment of log_in.html Template

```
<form method="post" class="form">
  {% csrf_token %}

  <div class="field">
    <i class="fa-regular␣fa-user"></i>
    <input type="text"
       name="username"
       placeholder="Username"
       autocomplete="off"
       required>
  </div>

  <div class="field">
    <i class="fa-solid␣fa-key"></i>
    <input type="password"
       name="password"
       placeholder="Password"
       autocomplete="new-password"
       required>
  </div>

  <button class="submit" type="submit">Login</button>

  {% if error %}
  <p class="error">{{ error }}</p>
  {% endif %}
</form>
```

The use of the `{ % csrf_token % }` tag ensures protection against cross-site request forgery attacks. The conditional block `{ % if error % }` supports informative user feedback when authentication fails.

Backend Authentication Logic

Authentication is implemented in `welcome/views.py` using Django's built-in authentication system. The system processes POST requests, validates credentials, and initializes an authenticated session.

Листинг 3: Login View Function in welcome/views.py

```
def log_in_f(request):
    error = None

    if request.method == "POST":
        username = request.POST.get("username", "").strip()
        password = request.POST.get("password", "")

        user = authenticate(request, username=username, password=password)

        if user is not None:
            login(request, user)
            return redirect('/main/')
        else:
            error = "Wrong␣login␣or␣password"

    return render(request, 'welcome/log_in.html', {'error': error})
```

The `authenticate` function verifies the provided credentials against stored user records. If authentication succeeds, the `login` function creates an active session for the user. In case of invalid credentials, the system returns a structured error message displayed by the template.

Routing and Navigation

After successful authentication, the system redirects the user to the main dashboard page using:

`return redirect('/main/')`

This behavior ensures a consistent user flow from public access pages to authenticated system functionality.

Requirements Coverage

The Login page implementation satisfies the following functional requirements:

- 4.1.1.3 – Allow users to log in using valid credentials.
- 4.1.1.5 – Display informative error messages in case of invalid authentication input.
- 4.1.2.1 – Provide access to the main dashboard page after successful authentication.

The implementation also supports the following non-functional requirements:

- 4.2.2.2 – The system shall enforce authentication mechanisms.
- 4.2.2.5 – Sensitive data is not exposed in system responses, as the error message does not reveal whether username or password is incorrect.
- 4.2.3.2 – Error messages are clear and informative.

Security Considerations

The login form includes CSRF protection and uses Django session management. Password verification is performed through Django's authentication subsystem, which supports cryptographic password hashing and secure credential comparison at the backend level. The system does not store or transmit passwords in plain text within application logic.

8.4 Main Page and Dashboard Structure Implementation

The Main page represents the central working interface of the system and is responsible for displaying analytical dashboards. After successful authentication, users are redirected to this page, which serves as the core environment for business data visualization and analysis.

Architectural Role of the Main Page

The Main page is designed as the primary container for dashboard components. While the current prototype includes placeholder blocks, the structure is prepared for integration of dynamic analytical visualizations such as:

- Key performance indicators
- Sales and supply charts
- Order statistics
- Inventory analytics

This page implements the functional requirements defined in Section 4.1.2 related to dashboard management.

Reusable Layout Template

To ensure consistent navigation across all internal system pages, a shared layout template is implemented in:

`main/templates/main/layout.html`

This template defines the sidebar and page structure, allowing all internal pages to inherit the same navigation panel.

Листинг 4: Fragment of layout.html

```
<aside>
    <img class="sidebar-logo"
         src="{% static 'main/img/logo_main.png' %}"
         alt="logo">

    <h3>Navigation</h3>
    <ul>
        <li><a href="{% url 'home' %}">
            <i class="fa-solid fa-house"></i>Home</a></li>
        <li><a href="{% url 'about' %}">
            <i class="fa-solid fa-info"></i>About</a></li>
        <li><a href="{% url 'logout' %}">
            <i class="fa-solid fa-door-closed"></i>Log out</a></li>
    </ul>
</aside>

<main>
    {% block content %}
    {% endblock %}
</main>
```

The `{ % block content % }` directive allows child templates to inject page-specific content while preserving the sidebar. This ensures structural consistency across all authenticated pages.

Sidebar Navigation Logic

The sidebar provides direct access to key application modules:

- Profile
- Home
- Orders
- Data
- About

- Log out

This persistent navigation supports usability requirements by ensuring intuitive system movement and minimizing user disorientation.

Styling and Visual Consistency

The visual appearance of the Main page and sidebar is controlled through:
`main/static/main/css/main.css`

Листинг 5: Fragment of main.css

```
aside {
    float: left;
    background: rgb(21, 56, 140);
    width: 15%;
    padding: 2.5%;
    height: 100vh;
    color: #fff;
}

aside ul li a {
    color: #fff;
    display: flex;
    align-items: center;
    gap: 10px;
}
```

The CSS ensures:

- Consistent branding through unified color scheme
- Fixed sidebar layout
- Responsive alignment of navigation elements

Requirements Coverage

The Main page implementation satisfies the following functional requirements:

- 4.1.2.1 – Provide access to the main dashboard page after authentication.
- 4.1.2.2 – Display analytical dashboards.
- 4.1.2.4 – Enable navigation between internal system pages.

It also supports non-functional requirements:

- 4.2.3.1 – Intuitive navigation.
- 4.2.3.3 – Consistent visual design.
- 4.2.6.1 – Architecture supporting future functional expansion.

Architectural Importance

The separation between the reusable layout template and page-specific content ensures:

- Maintainability
- Scalability
- Clear separation of structure and functionality

The Main page acts as the foundation for future integration of dynamic analytical dashboards and business intelligence components.

Conclusions

The developed prototype demonstrates that the fundamental architectural framework of the web application has been successfully implemented. The current version includes structured project organization, modular application separation, authentication functionality, persistent navigation via a reusable layout template, and a central dashboard container prepared for analytical content integration.

The system already ensures controlled user authentication, secure session handling, and consistent navigation across internal pages. The implemented layout architecture provides a stable foundation for future expansion and supports scalability and maintainability principles.

Further development stages will focus on:

- Full integration of database models for business data storage and management;
- Implementation of dynamic analytical dashboards on the Main page;
- Extension of data upload and validation mechanisms;
- Enhancement of role-based access control;
- Refinement and unification of visual design across all system pages.

Overall, the prototype confirms the feasibility of the selected technological stack and establishes a solid structural base for the implementation of analytical functionality and business intelligence components.

9 Graph Recommendation Module

9.1 Objective of the Module

The purpose of the graph recommendation module is to automatically suggest the most relevant data visualisations to a user after the upload of business data. The system provides a recommendation panel that displays charts suitable for the uploaded dataset based on data structure and availability.

The module does not generate arbitrary visualisations. Instead, it selects appropriate charts from a predefined catalogue of business-oriented templates. The recommended charts are presented to the user as suggestions, and the user may choose which visualisations to include in their main analytical dashboard.

The system aims to:

- recommend only charts supported by available data,
- avoid visually misleading or low-informative visualisations,
- prioritise commonly used and analytically valuable business charts,
- simplify dashboard creation for non-technical users.

9.2 Approach 1: Deterministic Ranking and Filtering

The first approach is based on deterministic ranking combined with data availability filtering. The system operates using a predefined library of chart templates commonly used in business analytics.

Each chart template contains metadata describing:

- required data types,
- minimum data volume,
- analytical category,
- expected business relevance.

After a dataset is uploaded, the system analyses its structure and determines which visualisations can be constructed without loss of interpretability or visual quality. Charts that do not meet minimum data requirements are excluded from recommendation.

The remaining charts are presented to the user as recommended visualisations. This approach ensures stability, transparency, and predictable system behaviour.

9.3 Data Profiling

Before generating recommendations, the system performs automatic data profiling. This stage extracts structural metadata from the uploaded dataset.

The profiling process includes:

- identification of column data types (numerical, categorical, datetime),

- calculation of dataset size,
- detection of missing values,
- estimation of category cardinality,
- identification of potential analytical dimensions.

The profiling process can be implemented using Python libraries such as `pandas` and `numpy`. Optionally, automated dataset analysis tools such as `ydata-profiling` may be used to accelerate metadata extraction.

9.4 Filtering Logic

Charts are filtered according to predefined feasibility rules. A chart is excluded if the dataset does not satisfy required structural conditions.

Typical filtering rules include:

- absence of required variable types,
- insufficient number of observations,
- excessive proportion of missing values,
- overly large categorical cardinality leading to unreadable plots.

Threshold values are manually calibrated during system design to ensure visual clarity and analytical usefulness.

Examples include:

- time-series charts require a sufficient number of temporal observations,
- pie charts require a limited number of categories,
- correlation visualisations require multiple numerical variables.

This filtering stage prevents the system from recommending charts that would appear visually incorrect or analytically meaningless.

9.5 Visualisation Technologies

The recommendation module operates on the backend using Python. Recommended visualisation tools include:

- `Plotly` for interactive visualisations,
- `Matplotlib` for static plotting,
- `Seaborn` for statistical visualisations.

Charts can be generated as JSON objects and transmitted to the Django frontend, where they are rendered dynamically within the user interface.

9.6 Approach 2: Agent-Based Recommendation

The second approach introduces an intelligent assistant responsible for recommending charts from the same predefined template database.

Instead of deterministic selection, the agent analyses dataset metadata and optionally considers user preferences expressed through natural language interaction.

Possible implementation is large language model (LLM) agent integrated via API.

Although this approach may improve interaction flexibility, it introduces higher computational cost and reduced predictability compared to deterministic methods.

9.7 Selected Approach

The deterministic ranking and filtering approach is selected as the primary implementation strategy.

This decision is motivated by:

- lower implementation complexity,
- predictable system behaviour,
- transparent recommendation logic,
- suitability for small business datasets,
- compatibility with project constraints.

The agent-based approach is considered a potential future extension rather than the core system component.

Список литературы

[1] Rodolfo Bonnin. *Machine Learning for Developers*. Packt Publishing, 2017. URL: https://books.google.ru/books?id=zRhKDwAAQBAJ.

[2] Maryam Ahmed Habeeb Omotunde. *A Comprehensive Review of Security Measures in Database Systems: Assessing Authentication, Access Control, and Beyond*. Mesopotamian Journal of CyberSecurit, 2023. URL: https://journals.mesopotamian.press/index.php/CyberSecurity/article/view/109.

[3] San Murugesan. *Web Application Development: Challenges and the Role of Web Engineering*. Springer, 2008. URL: https://link.springer.com/chapter/10.1007/978-1-84628-923-1_2.

[4] B. Sai Subrahmanya Tejesh. *Warehouse inventory management system using IoT and open source framework*. Alexandria Engineering Journal, 2018. URL: https://www.sciencedirect.com/science/article/pii/S1110016818301765.

[5] Rishi Vyas. *Comparative Analysis on Front-End Frameworks for Web Applications*. International Journal for Research in Applied Science Engineering Technology (IJRASET), 2022. URL: https://d1wqtxts1xzle7.cloudfront.net/89814198/Comparative_Analysis_on_Front_End_Frameworks_for_Web_Applications-libre.pdf.